TECHNICAL DESIGN DOCUMENT

Stock Transfer Order Management System

Full-stack implementation covering architecture, data model, API design, role-based access control, and the approval state machine.

React 18 + TypeScript | Vite + Tailwind CSS | Node.js + Express | SQLite (better-sqlite3) | JWT Auth | React Hook Form

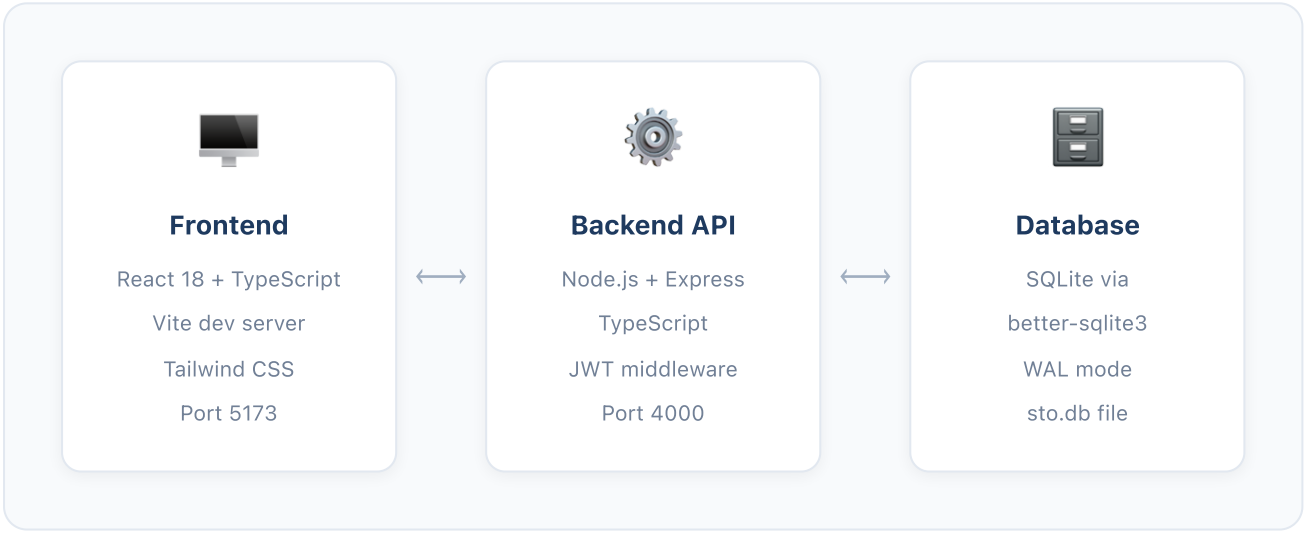
VERSION	DATE	AUTHOR	AUTH
1.0 — Demo Build	May 2026	Claude (Anthropic)	JWT / Demo Users → AD-ready

1 Overview

The STO Management System replaces an Excel-based tracker for pharmaceutical/supply chain Stock Transfer Orders. It digitises the full lifecycle — from a requestor submitting a material transfer request, through a multi-stage approval workflow (Inventory → Management → Finance), to logistics coordinating the physical shipment.

> **Core problem solved:** Paper/Excel-based STOs had no audit trail, no real-time visibility into approval status, and no role enforcement — anyone could change any field. This system enforces strict role-based access at every step, logs every action immutably, and surfaces actionable alerts to each role on their dashboard.

System Architecture



FRONTEND KEY LIBRARIES

- **React Router v6** — client-side routing with protected routes
- **React Hook Form** — form state, validation, conditional fields
- **Axios** — HTTP client with JWT header injection
- **Tailwind CSS** — utility-first styling, zero custom CSS files

BACKEND KEY LIBRARIES

- **Express 4** — HTTP server and routing
- **jsonwebtoken** — JWT sign & verify (8-hour expiry)
- **better-sqlite3** — synchronous SQLite driver
- **cors** — cross-origin requests from localhost:5173
- **dotenv** — environment config

2 Authentication & Role-Based Access

Authentication uses **JSON Web Tokens (JWT)**. On login the backend signs a token containing the user's ID, name, email, role, and plant. Every subsequent API request sends this token in the `Authorization: Bearer <token>` header. The `authenticate` middleware verifies the signature and attaches the decoded payload to `req.user`.

```
// middleware/auth.ts — JWT verification
export function authenticate(req: AuthRequest, res: Response, next: NextFunction) {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) { res.status(401).json({ message: 'No token' }); return; }
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET!) as JwtPayload;
    next();
  } catch { res.status(401).json({ message: 'Invalid token' }); }
}
```

Demo Users (AD-ready)

Seven demo users are hardcoded in `routes/auth.ts`. The JWT payload shape is already designed to match what Active Directory group-to-role mapping will produce — swapping the demo array for an LDAP call requires no other changes.

Username	Password	Role	Plant
admin	admin123	admin	ALL
requestor1	pass123	requestor	Plant A
requestor2	pass123	requestor	Plant B
inventory1	pass123	inventory_reviewer	ALL
manager1	pass123	management	ALL
finance1	pass123	finance	ALL
logistics1	pass123	logistics	ALL

Role Permissions

Requestor

Create & edit own STOs (DRAFT only). Submit for review. View own records only.

Inventory Reviewer

View all SUBMITTED/INVENTORY_REVIEW STOs. Approve or reject with notes.

Management

Approve/reject STOs in MANAGEMENT_REVIEW (auto-triggered by value thresholds).

Finance

Final approval/rejection in FINANCE_REVIEW. Controls APPROVED status gate.

Logistics

Updates shipment fields (STO#, tracking, PGI date, receipt). Drives SHIPPING → IN_TRANSIT → DELIVERED → CLOSED transitions.

Admin

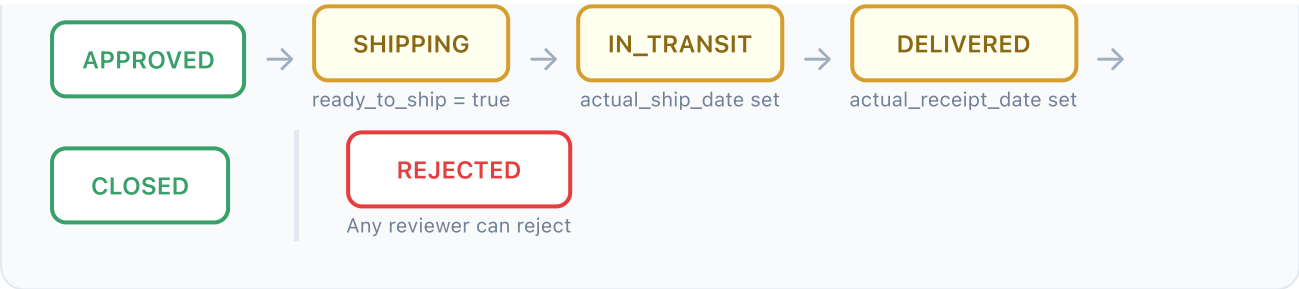
All permissions across all plants and all statuses. Can create, edit, approve at any stage.

Data Filtering by Role

The `GET /api/sto` endpoint applies a `WHERE` clause based on the requesting user's role before returning data — requestors only see their own records, inventory reviewers only see work in their queue, etc. There is no client-side filtering that could be bypassed.

3 Approval State Machine

Every STO moves through a defined set of statuses. Transitions are enforced server-side — a client can never jump a stage by calling the wrong endpoint.



Management Approval Auto-trigger

When an STO is created, the backend computes `management_approval_required` automatically based on configurable thresholds. This cannot be overridden by the requestor.

```
function needsManagementApproval(materialValue: number, freightCost: number): boolean {
  const matThreshold  = parseFloat(process.env.MANAGEMENT_APPROVAL_MATERIAL_THRESHOLD ||
'10000');
  const freightThreshold = parseFloat(process.env.MANAGEMENT_APPROVAL_FREIGHT_THRESHOLD
|| '5000');
  return materialValue > matThreshold || freightCost > freightThreshold;
}
```

Audit Log

Every status transition is written to `sto_audit_log` with the old status, new status, who performed the action, when, and optional notes. This is append-only — nothing in the codebase deletes from this table. The audit log is shown chronologically in the STO Detail view.

```
-- Every transition writes a row like this:
INSERT INTO sto_audit_log (sto_request_id, action, old_status, new_status,
                           performed_by, performed_by_name, notes)
```

```
VALUES (42, 'FINANCE_APPROVED', 'FINANCE_REVIEW', 'APPROVED',
        6, 'Sarah Davis', 'Verified cost centre allocation');
```

4 Database Design

Three tables. Originally designed for Microsoft SQL Server; migrated to SQLite (`better-sqlite3`) for local development with zero external dependencies. The schema is portable — moving back to SQL Server requires only changing column types (`INTEGER→BIT` , `TEXT→NVARCHAR` , `datetime('now')→GETDATE()`).

sto_requests — Main Table (45 columns)

Column Group	Key Fields	Notes
Identity	id, sto_id	sto_id format: STO-YYYY-00001 (auto-generated)
Request Info	request_date, priority, rush_request, public_holiday, repeat_shipment_calendar_year	Priority: 1=Urgent, 2=Expedited, 3=Standard
Sites	requesting_plant, shipping_site, receiving_site, toll_mfg	toll_mfg = contract manufacturing flag
Requestor	requestor_user_id, requestor_name, requestor_email	Denormalized from JWT at create time — survives AD migration
Material	material_sap, material_description, mpn_number, quantity, uom, batch_number, expiration_date	SAP material number is the master identifier
Shipping	shipping_conditions, container_information, controlled_shipping_required, brand_at_receiving_site	Cold chain / special handling flags
Financials	material_value, freight_cost, insurance_loss_required	Drives management approval auto-trigger
Scheduling	standard_estimated_ship_date, expedited_estimated_ship_date, receiving_site_need_by_date, estimated_ship_by_date	Expedited date only relevant when rush=true
Approvals	inventory_approved, management_approved, finance_approved + _by + _at + _notes variants	Each approval is a separate column set
Shipment	sto_number, shipment_id, tracking_id, pgi_date, actual_ship_date, actual_receipt_date, ready_to_ship, delivery_closed_out	Updated by logistics role
Status	status, rejection_reason, created_at, updated_at	updated_at set on every write

sto_audit_log — Immutable Append-only Log

Column	Type	Purpose
id	INTEGER PK	Auto-increment row identifier
sto_request_id	INTEGER	FK → sto_requests.id
action	TEXT	CREATED, SUBMITTED, INVENTORY_APPROVED, INVENTORY_REJECTED, MANAGEMENT_APPROVED, MANAGEMENT_REJECTED, FINANCE_APPROVED, FINANCE_REJECTED, SHIPMENT_UPDATED
old_status / new_status	TEXT	State before and after the action
performed_by / performed_by_name	INT / TEXT	User ID and name (name denormalized so it's readable after AD changes)
notes	TEXT	Reviewer comments or rejection reasons
performed_at	TEXT	SQLite datetime string, set by default to datetime('now')

sto_config — Key/Value Config

Stores the approval thresholds. Pre-seeded at schema creation. Editable by admin without redeploying. Currently two rows: `management_approval_material_threshold` (default 10000) and `management_approval_freight_threshold` (default 5000).

Why no users table?

> User info (name, email, role, plant) is deliberately denormalized into `sto_requests` at write time from the JWT payload. This means all records remain readable and self-contained after Active Directory migration, even if user IDs change between systems.

5 API Design

RESTful API served at `http://localhost:4000/api`. All routes except `/auth/login` and `/auth/demo-users` require a valid JWT. Role enforcement happens inside each route handler — not just at the middleware level.

Method	Route	Role(s)	What it does
POST	/auth/login	Public	Returns JWT + user payload
GET	/auth/me	Any	Returns current user from JWT
GET	/auth/demo-users	Public	Returns demo user list for login page
GET	/sto	All	List STOs, role-filtered, supports ?status=&priority=&search=
POST	/sto	requestor, admin	Create STO — saves as DRAFT, auto-sets sto_id and management flag
GET	/sto/:id	All	Full STO detail + audit_log array
PUT	/sto/:id	requestor (own), admin	Update DRAFT STO only
POST	/sto/:id/submit	requestor, admin	DRAFT → SUBMITTED
POST	/sto/:id/inventory	inventory_reviewer, admin	{ approved, notes } → MANAGEMENT_REVIEW or FINANCE_REVIEW or REJECTED
POST	/sto/:id/management	management, admin	{ approved, notes } → FINANCE_REVIEW or REJECTED
POST	/sto/:id/finance	finance, admin	{ approved, notes } → APPROVED or REJECTED
POST	/sto/:id/shipment	logistics, admin	Updates shipment fields, auto-advances status on field presence
GET	/health	Public	Returns { status: "ok", timestamp }

Shipment Status Auto-advance Logic

The `/shipment` endpoint is smart — it advances the status automatically based on which fields are present in the request body, without requiring the client to specify the target status:

```
let newStatus = sto.status;
if (body.ready_to_ship && sto.status === 'APPROVED')     newStatus = 'SHIPPING';
```

```
if (body.actual_ship_date && sto.status === 'SHIPPING')   newStatus = 'IN_TRANSIT';
if (body.actual_receipt_date && sto.status === 'IN_TRANSIT') newStatus = 'DELIVERED';
if (body.delivery_closed_out)                              newStatus = 'CLOSED';
```

Frontend API Client

An Axios instance in `src/api/client.ts` automatically attaches the JWT from `localStorage` to every request. A response interceptor clears the token and redirects to `/login` on 401 responses.

```
const api = axios.create({ baseURL: 'http://localhost:4000/api' });

api.interceptors.request.use(cfg => {
  const token = localStorage.getItem('sto_token');
  if (token) cfg.headers.Authorization = `Bearer ${token}`;
  return cfg;
});
```

6 Frontend Architecture

Routing & Protected Routes

React Router v6 handles navigation. A `ProtectedRoute` wrapper checks for an authenticated user from `AuthContext`. Unauthenticated requests redirect to `/login`. After login the user is redirected to `/dashboard`.

Route	Component	Who can access
/login	Login.tsx	Public (redirects to /dashboard if already logged in)
/dashboard	Dashboard.tsx	All authenticated users
/sto	STOList.tsx	All authenticated users (data is role-filtered by API)
/sto/new	STOForm.tsx	requestor, admin
/sto/:id	STODetail.tsx	All (requestor sees own only; API enforces this)
/sto/:id/edit	(planned)	requestor (own DRAFT), admin

AuthContext

A React context provides `user`, `login()`, and `logout()` globally. On app load, it attempts to restore a session from `localStorage` by calling `GET /api/auth/me` with the stored token. If the token is expired or invalid, the user is silently logged out.

Component Breakdown

DASHBOARD.TSX

Role-specific alert banners ("X STOs awaiting your action"), summary stat cards (Total / Pending / In Transit / Closed), and a recent requests table. All data from `GET /api/sto` — the API handles role filtering so the component needs no conditional data logic.

STOLIST.TSX

Filterable, searchable table of all accessible STOs. Filter bar: status dropdown, priority dropdown, free-text search (queries API with `?status=&priority=&search=`). Rows are clickable links to STODetail. Status and priority are coloured badges.

STOFORM.TSX (CREATE)

Multi-section form using React Hook Form. Four sections: Request Info, Material Details, Financial Info, Scheduling. Smart behaviour: Expedited

STODETAIL.TSX

Displays all STO fields in grouped cards. Shows inline approval action panels for the relevant role (yellow "Action Required" → approve/reject with notes).

Ship Date field only appears when Rush Request is checked. Management-approval warning banner activates live as the user types values exceeding the thresholds.

Logistics role sees the shipment update form. Includes the full audit log at the bottom.

LAYOUT.TSX

Shared wrapper: navigation bar with links to Dashboard, All STOs, and (for admin) Admin panel. Shows logged-in user name and role badge. Logout button clears localStorage and redirects to /login.

STATUSBADGE / PRIORITYBADGE

Shared badge components. StatusBadge maps each status string to a colour: DRAFT=grey, SUBMITTED=blue, *_REVIEW=yellow, APPROVED=green, SHIPPING/IN_TRANSIT=teal, DELIVERED=indigo, CLOSED=grey, REJECTED=red. PriorityBadge: 1=red, 2=orange, 3=green.

Form Smart Logic (STOForm.tsx)

```
// Rush Request checkbox unlocks Expedited ESD field
const rushRequest = watch('rush_request');
{rushRequest && (
  <Field label="Expedited Estimated Ship Date">
    <input type="date" {...register('expedited_estimated_ship_date')} />
  </Field>
)}

// Live management approval warning banner
const materialValue = parseFloat(String(watch('material_value') || '0'));
const freightCost   = parseFloat(String(watch('freight_cost')   || '0'));
const willNeedMgmt = materialValue > 10000 || freightCost > 5000;
```

7 Key Design Decisions

1 SQLite for development, SQL Server for production

The original schema targeted Microsoft SQL Server. For local development, we swapped to SQLite via `better-sqlite3` to eliminate the external DB dependency. The `better-sqlite3` synchronous API actually simplified route code — no more pool/request boilerplate. Moving back to SQL Server is a 3-file change (connection.ts + type casting in routes).

2 Demo users in-memory, not in DB

Avoiding the chicken-and-egg problem: you need to log in to set up the DB, but you need the DB to log in. Demo users live in `routes/auth.ts` as an array. The login page fetches and displays them so any tester can click to autofill credentials without reading docs.

3 User info denormalized into STO records

Requestor name and email are copied into `sto_requests` from the JWT at creation time. This means records stay readable after AD migration even if the user's ID changes between systems. It also means no JOIN is needed to show who created each STO.

4 Management approval computed server-side, not set by requestor

The requestor form has no "Management Approval Required" checkbox. The flag is computed in the POST handler using thresholds from environment variables. This prevents requestors from gaming the system. Thresholds are readable from `sto_config` and will be editable via the planned admin panel without redeployment.

5 Audit log on every transition, append-only

Every `POST /sto/:id/*` endpoint writes a row to `sto_audit_log` capturing old status, new status, who acted, when, and notes. Nothing in the codebase issues a DELETE or UPDATE on this table. This gives a complete, tamper-evident history for compliance and dispute resolution.

6 JWT payload mirrors AD group shape

The token payload (`userId` , `username` , `role` , `name` , `email` , `plant`) is already designed to match what an LDAP/passport-azure-ad call would produce after mapping AD group names to role strings. Replacing the demo user array with an AD call requires no changes to middleware, routes, or frontend.

7 Boolean normalization for SQLite compatibility

SQLite stores booleans as `0` / `1` integers. A `normalizeSto()` helper converts known bit columns to JavaScript booleans before sending JSON to the frontend. This keeps the frontend type definitions correct and ensures "Yes/No" displays correctly in the detail view without changing the frontend code.

What's Planned Next

Feature	Where to implement	Complexity
Edit STO page (/sto/:id/edit)	New STOEditForm.tsx page, re-use STOForm fields, call PUT /sto/:id	Low
Admin panel	New AdminPanel.tsx — user list + threshold editor (reads/writes sto_config)	Medium
Rush SLA tracking (15 days)	Dashboard alert: compare request_date + 15 days vs today for rush STOs in review	Low

Feature	Where to implement	Complexity
Email notifications	nodemailer hook in approvals.ts after each updateStatus() call	Medium
Pagination	Add LIMIT/OFFSET to GET /api/sto; add page controls to STOList.tsx	Low
PDF export	Client-side: react-pdf or jsPDF reading the STO detail JSON	Medium
Active Directory auth	Replace DEMO_USERS array in auth.ts with ldapjs or passport-azure-ad call. Map AD group → role string.	High

8 Setup & Running

Prerequisites

- Node.js v18+ (v26 confirmed working)
- npm v9+
- No external database required — SQLite DB is created automatically at `backend/sto.db` on first start

Start the Backend

```
cd /Users/anjalikumari/Documents/sto-management/backend
npm install
npm run dev
# → STO backend running on http://localhost:4000
# → sto.db created automatically if it doesn't exist
```

Start the Frontend

```
cd /Users/anjalikumari/Documents/sto-management/frontend
npm install
npm run dev
# → http://localhost:5173
```

Environment Variables (backend/.env)

Variable	Default	Purpose
JWT_SECRET	dev-secret	Signs and verifies JWTs — change to a long random string in production
PORT	4000	Backend HTTP port
MANAGEMENT_APPROVAL_MATERIAL_THRESHOLD	10000	USD value above which management sign-off is required
MANAGEMENT_APPROVAL_FREIGHT_THRESHOLD	5000	Freight cost above which management sign-off is required

> **Migrating back to SQL Server:** (1) Restore `mssql` in connection.ts using the original pool/request pattern. (2) Run `backend/src/db/schema.sql` against your SQL Server instance. (3)

Update `.env` with DB_SERVER, DB_DATABASE, DB_USER, DB_PASSWORD. No route files or frontend changes needed.

Project File Map

```
sto-management/
├── backend/
│   ├── src/
│   │   ├── index.ts                 Express entry point, CORS, route mounting
│   │   ├── types/index.ts           STORequest, STOStatus, User, JwtPayload
│   │   ├── db/
│   │   │   ├── connection.ts        SQLite init, schema creation, db singleton
│   │   │   └── schema.sql           Original SQL Server DDL (reference)
│   │   ├── middleware/
│   │   │   └── auth.ts              JWT verify, requireRole() helper
│   │   └── routes/
│   │       ├── auth.ts              POST /login, GET /me, GET /demo-users
│   │       ├── sto.ts               GET / POST / PUT /sto and /sto/:id
│   │       └── approvals.ts         POST
/sto/:id/{submit,inventory,management,finance,shipment}
│   ├── sto.db                       SQLite database (auto-created)
│   └── .env                         Environment config
└── frontend/
    └── src/
        ├── App.tsx                  Router + ProtectedRoute
        ├── context/AuthContext.tsx  Login state, token storage, /me on load
        ├── api/client.ts            Axios instance with JWT interceptor
        ├── components/
        │   ├── Layout.tsx           NavBar + page wrapper
        │   └── StatusBadge.tsx      Status + Priority coloured badges
        └── pages/
            ├── Login.tsx            Login form + clickable demo user panel
            ├── Dashboard.tsx        Role-specific alerts + stats + recent STOs
            ├── STOList.tsx          Filterable table (status, priority, search)
            ├── STOForm.tsx          Full STO creation form with smart fields
            └── STODetail.tsx        View + inline approval actions + audit log
```